UNIVERSITÉ JOSEPH KI-ZERBO

(UJKZ)

★ ★ ★ ★ ★

INSTITUT BURKINABÈ DES ARTS ET MÉTIERS

(IBAM)

Cours : Développement à base de composants et services Web

PROJET : API REST DE TRANSACTIONS BANCAIRES

Express.js ▪ KNIME ▪ Prometheus ▪ Grafana

Données bancaires fictives : contrôle, API REST et supervision

Thème 6 ▪ Groupe 5

Filière : Master 1 SIE

Présenté par

BANDE Hamidou
SOMBIE Roseline
KOUMTOUBRE Davy
BADO Lionel

Encadré par

KI Fadel Omar

Dépôt : `https://github.com/BANDE-Hamidou/API_REST_Transaction`

Année universitaire 2025–2026

Table des matières

Sigles et abréviations **i**

1 Contexte et exigences **1**

1.1 Le sujet . . . 1
1.2 Exigences de l'exercice 6 . . . 1
1.3 Exigences communes à tous les groupes . . . 1
1.4 Hypothèses retenues . . . 1

2 Architecture et choix techniques **2**

2.1 Vue d'ensemble . . . 2
2.2 Composants et justification . . . 3
2.3 Organisation du code de l'API . . . 3
2.4 Ports . . . 4

3 Données et base PostgreSQL **4**

3.1 Des données fictives, piégées exprès . . . 4
3.2 Le schéma de la base . . . 4
3.3 Trois barrières de qualité . . . 5
3.4 Un chargement idempotent et transactionnel . . . 5

4 Workflow KNIME de contrôle et de transformation **6**

4.1 Rôle dans la chaîne . . . 6
4.2 Les étapes du workflow . . . 7
4.3 Les règles de contrôle (nœud 9) . . . 8
4.4 Résultat obtenu . . . 8
4.5 Vérification automatique . . . 10
4.6 Difficultés rencontrées . . . 10

5 API REST de transactions **10**

5.1 Responsabilités et organisation . . . 11
5.2 Routes et opérations . . . 11
5.3 Filtres, pagination et tri . . . 11
5.4 Validation et réponses d'erreur . . . 11
5.5 Sécurité des requêtes SQL . . . 12
5.6 Vérification . . . 12

6 Supervision avec Prometheus et Grafana **12**

6.1 Les métriques exposées . . . 12

6.2 Prometheus . . . 13
6.3 Grafana . . . 14
6.4 Panne contrôlée de la base . . . 14

7 Tests et résultats **15**
7.1 Stratégie . . . 15
7.2 Le test de bout en bout . . . 16
7.3 Résultats obtenus . . . 16

8 Installation et reproductibilité **17**
8.1 Objectif . . . 17
8.2 Prérequis . . . 18
8.3 Les étapes . . . 18
8.4 Pièges constatés sur les postes de l'équipe . . . 19
8.5 Test sur un poste neuf . . . 19

9 Limites et perspectives **19**
9.1 Périmètre fonctionnel et données . . . 20
9.2 Sécurité et exploitation . . . 20
9.3 Évolutions possibles . . . 20

A Annexes **20**
A.1 Contrat de données d'une transaction . . . 20
A.2 Organisation du dépôt . . . 21

Table des figures

2.1 Architecture de la chaîne . . . 2

3.1 Les trois tables créées au démarrage de PostgreSQL . . . 5
3.2 Un montant négatif est refusé par la contrainte `transactions_montant_check` . . . 5
3.3 Deuxième chargement : aucune insertion, 946 mises à jour, 64 rejets . . . 6

4.1 Le workflow complet : 23 nœuds exécutés (feux verts) . . . 7
4.2 Configuration du nœud 9 : les 30 règles de contrôle . . . 8
4.3 Sortie du nœud 23 : 18 motifs de rejet, 64 lignes au total . . . 9
4.4 La sortie de KNIME est conforme au résultat attendu . . . 10

6.1 Prometheus : la cible `api-transactions` est UP . . . 13
6.2 Prometheus : les trois règles d'alerte, inactives . . . 13
6.3 Le tableau de bord pendant le trafic de démonstration . . . 14
6.4 Grafana pendant la panne de PostgreSQL : réponses 503 et délais de connexion . . . 15

7.1 Test de bout en bout : 7 vérifications réussies 16
7.2 Collection Postman jouée avec Newman : 15 requêtes, 15 assertions, 0 échec 17

Liste des tableaux

1.1 Exigences de l'exercice 6 et chapitre qui y répond 1
1.2 Exigences communes 1

2.1 Composants, rôles et justification 3
2.2 Organisation de `api/src` 3
2.3 Ports des services (modifiables dans `.env`) 4

3.1 Composition du jeu de données 4
3.2 Idempotence du chargement (poste de BANDE, 28/10/2026) 6

4.1 Les six étapes du workflow KNIME 7
4.2 Points de contrôle du workflow (poste de SOMBIE, 29/10/2026) 9
4.3 Rejets par motif produits par KNIME 9
4.4 Difficultés rencontrées lors de la construction du workflow 10

5.1 Principales routes de l'API 11

6.1 Métriques de l'API : requêtes, erreurs et temps de réponse (exigence E4) 12
6.2 Règles d'alerte (`monitoring/prometheus/alert_rules.yml`) 13

7.1 Les suites de tests 15
7.2 Résultats des tests 17

8.1 Outils à installer sur le poste 18
8.2 Arrêter, relancer, réinitialiser 18
8.3 Problèmes rencontrés et corrections 19
8.4 Déroulé du test d'installation sur un clone neuf (poste de BANDE, 29/10/2026) . . 19

A.1 Contrat de données (`docs/CONTRATS.md`) 21

Sigles et abréviations

Sigle	Signification
API	Application Programming Interface (interface de programmation)
CHECK	Contrainte SQL qui refuse une ligne ne respectant pas une condition
CRUD	Create, Read, Update, Delete (créer, lire, modifier, supprimer)

Sigle	Signification
CSV	Comma-Separated Values (valeurs séparées par des virgules)
E2E	End-to-End (test de bout en bout)
ETL	Extract, Transform, Load (extraire, transformer, charger)
HTTP	HyperText Transfer Protocol
JDBC	Java Database Connectivity
JSON	JavaScript Object Notation
KNIME	Konstanz Information Miner (plateforme d'analyse de données)
OpenAPI	Format standard de description d'une API REST
p50, p95, p99	50e, 95e et 99e centiles du temps de réponse
PR	Pull Request (demande de fusion d'une branche, sur GitHub)
PromQL	Prometheus Query Language (langage de requête de Prometheus)
REST	Representational State Transfer
SQL	Structured Query Language
UTF-8	Encodage des caractères Unicode sur 8 bits
XOF	Franc CFA de l'Afrique de l'Ouest (code ISO 4217)

1. Contexte et exigences

1.1 Le sujet

Le thème 6 du projet *Développement à base de composants et services Web* demande une API REST de transactions bancaires construite avec Express.js, alimentée par un workflow KNIME et supervisée par Prometheus et Grafana. L'enjeu n'est pas une seule technologie mais leur assemblage : chaque composant a un rôle distinct et doit pouvoir être remplacé ou testé seul.

Notre chaîne fait cinq choses :

1. **générer** 1010 transactions bancaires fictives, dont des erreurs injectées exprès ;
2. **contrôler et transformer** ces données avec KNIME : 946 lignes valides, 64 rejetées avec leur motif ;
3. **charger** le résultat dans PostgreSQL, sans jamais créer de doublon ;
4. **exposer** les transactions par une API REST filtrable, paginée et modifiable ;
5. **mesurer** les requêtes, les erreurs et les temps de réponse de l'API.

1.2 Exigences de l'exercice 6

Tableau 1.1 – Exigences de l'exercice 6 et chapitre qui y répond

N°	Travail demandé (énoncé)	Chapitre
E1	Générer des transactions bancaires fictives	3
E2	Construire un workflow KNIME pour contrôler et transformer les données	4
E3	Développer l'API REST et ses filtres	5
E4	Exposer les métriques de requêtes, d'erreurs et de temps de réponse	6
E5	Configurer Prometheus et Grafana	6
E6	Utiliser Cypress si une interface web est fournie ; sinon, un outil adapté aux tests d'API	7

1.3 Exigences communes à tous les groupes

Tableau 1.2 – Exigences communes

N°	Exigence
G1	Dépôt Git public : code et dépendances, scripts de création des bases, configurations, workflows et contrats d'API, README (architecture, prérequis, installation), tests et résultats attendus, présentation PowerPoint.
G2	Installation reproductible sur une machine neuve ; Docker Compose pour les services compatibles ; procédure distincte pour les outils à installation particulière (KNIME).
G3	Au minimum un scénario fonctionnel réussi et un test de bout en bout de la chaîne principale.

1.4 Hypothèses retenues

L'énoncé laisse plusieurs points ouverts. Nos choix, justifiés au chapitre 2 :

— la base n'est pas imposée : nous utilisons **PostgreSQL 16** ;
— « l'API et ses filtres » : consultation filtrée *et* création, modification, suppression ;
— KNIME écrit deux fichiers CSV, vérifiés puis chargés par un chargeur idempotent, plutôt que d'écrire directement dans la base ;
— aucune interface web n'est demandée : les tests utilisent Jest, Supertest, Postman (Newman) et un test de bout en bout, pas Cypress ;
— l'authentification est hors périmètre (piste d'amélioration au chapitre 9).

2. Architecture et choix techniques

2.1 Vue d'ensemble

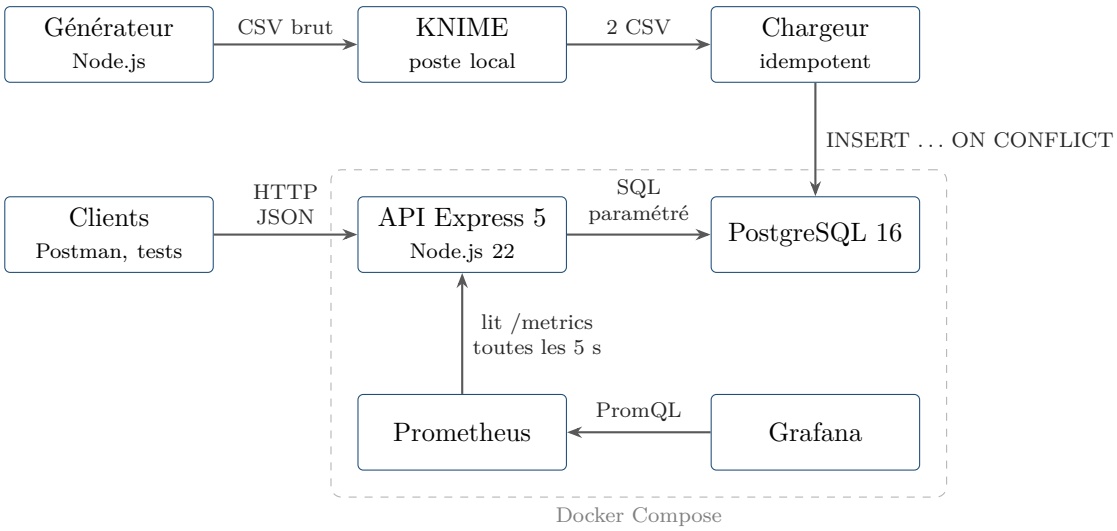


FIGURE 2.1 – Architecture de la chaîne

KNIME est une application de bureau : il tourne hors de Docker, sur le poste chargé de l'ETL. Les quatre autres services sont décrits dans `docker-compose.yml`. Le dossier `data/` est monté en lecture seule dans le conteneur de l'API : KNIME y écrit, le chargeur y lit.

2.2 Composants et justification

Tableau 2.1 – Composants, rôles et justification

Composant	Rôle	Pourquoi ce choix
Générateur (Node.js)	transactions fictives et anomalies contrôlées	déterministe (graine 2026) : toute l'équipe a les mêmes données
KNIME (exigé)	normaliser, contrôler, dédoublonner, enrichir	workflow visuel, explicable nœud par nœud
Oracle ETL (Node.js)	résultat attendu des règles KNIME	permet de *prouver* que la sortie KNIME est juste
PostgreSQL 16	stockage	contraintes `CHECK`, transactions, `ON CONFLICT`, index
Express.js 5 (exigé)	API REST	transmet les erreurs des fonctions asynchrones au gestionnaire d'erreurs
prom-client 15	exposition de `/metrics`	bibliothèque de référence pour Node.js
Prometheus (exigé)	collecte et alertes	modèle *pull* : l'API ne dépend pas de Prometheus
Grafana (exigé)	tableau de bord	provisionné par fichiers : aucun clic, installation reproductible
Jest, Supertest, Newman	tests	pas d'interface web, donc des outils de test d'API (exigence E6)

2.3 Organisation du code de l'API

Tableau 2.2 – Organisation de `api/src`

Dossier ou fichier	Contenu
`server.js`, `app.js`, `config.js`	démarrage, arrêt propre, assemblage d'Express, variables d'environnement
`domain/rules.js`	règles métier : valeurs autorisées, formats, catégories de montant
`validation/`	validation des corps (POST, PATCH) et des paramètres de requête
`repositories/`	accès à PostgreSQL ; `queryBuilder` produit le SQL paramétré
`routes/`	transactions, rejets, chargements
`metrics/metrics.js`	compteurs, histogramme et middleware de mesure
`middleware/errorHandler.js`	format d'erreur unique (400, 404, 409, 503, 500)
`scripts/load-csv.js`	chargement des sorties KNIME

Les routes ne parlent qu'à une *interface* de dépôt de données. En production, c'est PostgreSQL ; dans les tests, un dépôt en mémoire de même interface. On teste ainsi le HTTP sans base, puis le SQL séparément.

2.4 Ports

Tableau 2.3 – Ports des services (modifiables dans `.env`)

Service	Sur le poste	Dans le réseau Docker
API	`http://localhost:3000`	`api:3000`
PostgreSQL	`localhost:5432`	`postgres:5432`
Prometheus	`http://localhost:9090`	`prometheus:9090`
Grafana	`http://localhost:3001`	`grafana:3000`

3. Données et base PostgreSQL

3.1 Des données fictives, piégées exprès

Le générateur produit 1000 transactions et ajoute 10 doublons de référence, soit 1010 lignes.
— programme : `data/scripts/generate-transactions.js` ;
— fichier produit : `data/raw/transactions_brutes.csv` ;
— liste des anomalies injectées : `data/raw/anomalies_injectees.json`.
Il utilise une graine fixe (2026) : relancé sur n'importe quel poste, il redonne exactement le même fichier. Le résultat attendu de l'ETL est donc connu à la ligne près.

Une chaîne de contrôle qui ne reçoit que des données propres ne prouve rien. Le générateur injecte donc deux sortes d'anomalies, chacune listée avec le motif attendu :
— **54 anomalies bloquantes** : la donnée est absente, fausse ou ambiguë (montant négatif, devise `FCFA`, date impossible, référence mal formée…). La ligne est rejetée avec un motif ;
— **26 anomalies corrigeables** : l'intention est certaine (minuscules, espaces, virgule décimale, séparateur de milliers, date au format français). La ligne est corrigée puis acceptée.

Tableau 3.1 – Composition du jeu de données

Catégorie	Lignes
Lignes brutes	1010
dont doublons de référence	10
dont anomalies bloquantes	54
dont anomalies corrigeables (acceptées après correction)	26
Lignes valides attendues	946
Lignes rejetées attendues (54 + 10)	64

Un ETL de référence écrit en JavaScript (`data/scripts/reference-etl.js`) applique les mêmes règles que KNIME et produit le résultat attendu dans `data/expected/`. Il sert d'oracle : le comparateur `npm run data:compare` confronte la sortie de KNIME à ce résultat, ligne par ligne (chapitre 4).

3.2 Le schéma de la base

Le script `db/init/01_schema.sql` est exécuté automatiquement par PostgreSQL à son premier démarrage. Il crée trois tables :

— `transactions` : les transactions valides, venues de KNIME (`source = 'KNIME'`) ou de l'API (`source = 'API'`) ;
— `transactions_rejetees` : les lignes rejetées, stockées en texte brut avec leur `motif_rejet` ;
— `chargements` : le journal de chaque chargement (lignes lues, insérées, mises à jour, rejetées).



FIGURE 3.1 – Les trois tables créées au démarrage de PostgreSQL

La table `transactions` porte les règles métier sous forme de contraintes : format de la référence (`^TXN-[0-9]{6}$`) et des comptes, valeurs autorisées pour le type, la devise, le statut et le canal, `montant > 0`, compte destination obligatoire pour un virement et différent du compte source. Six index accélèrent les filtres de l'API (date, type, statut, comptes, montant).

Ces contraintes sont une **dernière barrière** : même un `INSERT` écrit à la main est refusé.

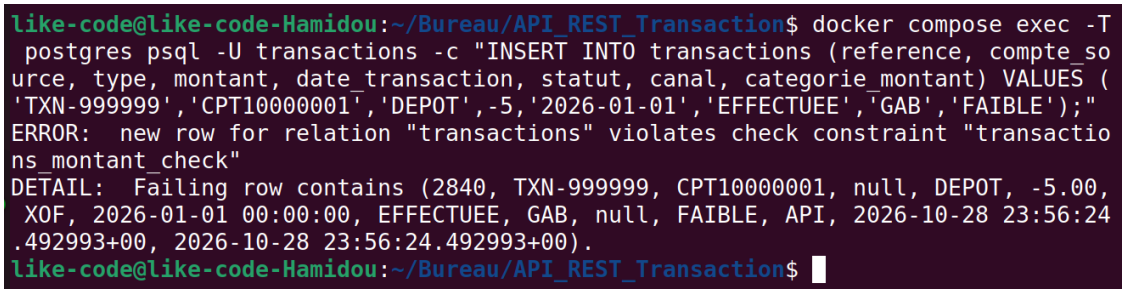


FIGURE 3.2 – Un montant négatif est refusé par la contrainte `transactions_montant_check`

3.3 Trois barrières de qualité

1. **KNIME** applique toutes les règles et motive chaque rejet ;
2. **le chargeur** revalide chaque ligne « valide » avec les règles de l'API ; une ligne refusée à ce stade est enregistrée avec le motif `REJET_CHARGEMENT` (cela ne doit pas arriver) ;
3. **PostgreSQL** refuse toute ligne qui viole une contrainte.

3.4 Un chargement idempotent et transactionnel

Le chargeur `api/src/scripts/load-csv.js` lit les deux fichiers produits par KNIME et les charge dans **une seule transaction SQL** : soit tout est chargé, soit rien. Il utilise `INSERT ... ON CONFLICT (reference) DO UPDATE` : relancer le chargement met à jour les lignes existantes au lieu de créer des doublons. Une transaction créée par l'API n'est jamais écrasée par un rechargement KNIME.

```
like-code@like-code-Hamidou:~/Bureau/API_REST_Transaction$ docker compose exec -T
 api npm run load -- /data/expected

> api-transactions@1.0.0 load
> node src/scripts/load-csv.js /data/expected

[chargement] lecture de /data/expected
[chargement] terminé : {
  "chargement_id": 4,
  "valides_lues": 946,
  "inserees": 0,
  "mises_a_jour": 946,
  "ignorees_source_api": 0,
  "rejetees": 64,
  "rejets_au_chargement": 0,
  "ecarts_categorie": 0
}
like-code@like-code-Hamidou:~/Bureau/API_REST_Transaction$
```

FIGURE 3.3 – Deuxième chargement : aucune insertion, 946 mises à jour, 64 rejets

TABLEAU 3.2 – Idempotence du chargement (poste de BANDE, 28/10/2026)

Chargement	Insérées	Mises à jour	Rejetées
1er passage	946	0	64
2e passage	0	946	64

4. Workflow KNIME de contrôle et de transformation

4.1 Rôle dans la chaîne

Le workflow KNIME répond à l'exigence E2 : contrôler et transformer les données brutes avant leur chargement. Il lit `data/raw/transactions_brutes.csv` (1010 lignes) et écrit deux fichiers dans `data/processed/` :

— `transactions_valides.csv` : 946 lignes, plus la colonne `categorie_montant` ;
— `transactions_rejetees.csv` : 64 lignes, chacune avec son `motif_rejet`.

KNIME écrit des CSV plutôt que d'écrire directement dans PostgreSQL. Aucun pilote JDBC n'est à configurer sur le poste, et la sortie peut être vérifiée automatiquement *avant* d'être chargée (chapitre 3).

Le workflow a été construit avec KNIME Analytics Platform 5.12, sans extension, en suivant la spécification `docs/KNIME_WORKFLOW.md`. Il est exporté dans `knime/transactions_etl.knwf`.

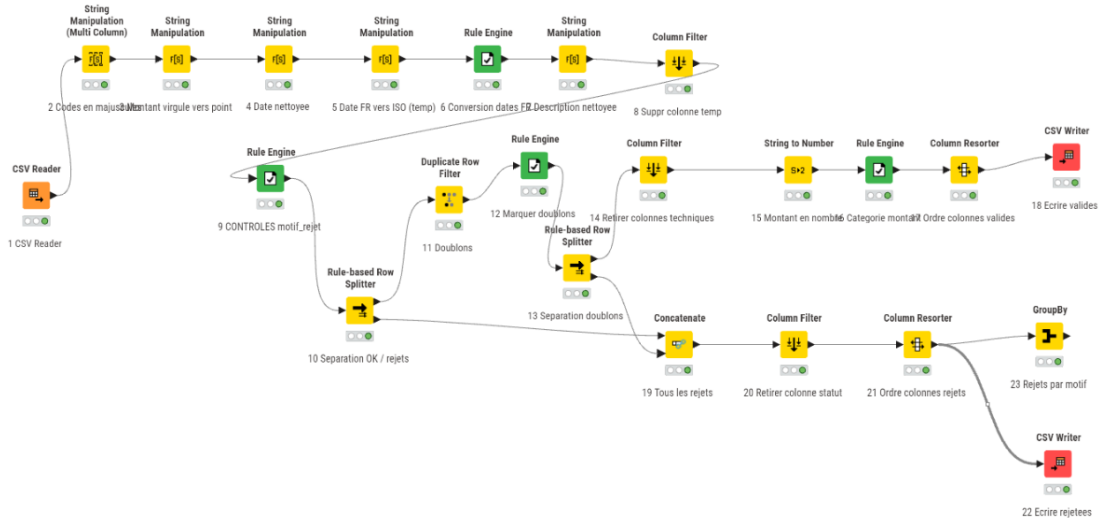


FIGURE 4.1 – Le workflow complet : 23 nœuds exécutés (feux verts)

4.2 Les étapes du workflow

Les 23 nœuds forment six étapes. Chaque nœud porte son numéro dans son nom, pour que la capture se lise dans l'ordre.

TABLEAU 4.1 – Les six étapes du workflow KNIME

Nœuds	Étape	Ce qui est fait
1	Lire	CSV Reader ; **toutes les colonnes forcées en texte** (String)
2–8	Normaliser	codes en majuscules sans espaces ; montant : espaces retirés, virgule → point ; date française `JJ/MM/AAAA hh:mm:ss` → ISO `AAAA-MM-JJThh:mm:ss`
9–10	Contrôler	Rule Engine : une colonne `motif_rejet` ; séparation des lignes `OK` et des rejets
11–13	Dédoublonner	doublons sur `reference` seule ; la première occurrence est gardée, les suivantes reçoivent `DOUBLON_REFERENCE`
14–18	Enrichir et écrire	montant converti en nombre ; `categorie_montant` ; ordre des colonnes ; écriture des valides
19–23	Écrire les rejets	réunion des deux sorties de rejet ; écriture ; comptage par motif

Pourquoi tout lire en texte. Si le CSV Reader devine les types, il lit `montant` comme un nombre : les valeurs `"mille"` ou `"12500,00"` deviennent des valeurs manquantes. On ne pourrait plus dire *pourquoi* la ligne est fausse. Le montant n'est converti en nombre qu'au nœud 15, une fois la ligne contrôlée.

Corriger ou rejeter. Une anomalie dont l'intention est certaine est corrigée : minuscules, espaces, virgule décimale, séparateur de milliers, date au format français. Les 26 anomalies de ce type sont acceptées après correction. Une donnée absente, fausse ou ambiguë est rejetée avec un motif.

4.3 Les règles de contrôle (nœud 9)

Le nœud 9 est le cœur du workflow. Ses 30 règles sont évaluées dans l'ordre et la première règle vraie donne le motif. Elles sont la traduction exacte de la fonction `motifRejet` de `data/scripts/rules.js`, dans le même ordre : c'est ce qui permet de comparer KNIME à l'ETL de référence.

Extrait 4.1 – Début des règles du nœud 9 (Rule Engine)

```
MISSING $reference$ => "REFERENCE_MANQUANTE"
$reference$ = "" => "REFERENCE_MANQUANTE"
NOT $reference$ MATCHES "TXN-[0-9]{6}" => "REFERENCE_INVALIDE"
...
$montant$ LIKE "-*" => "MONTANT_NEGATIF"
NOT $montant$ MATCHES "[0-9]+([.][0-9]{1,2})?" => "MONTANT_NON_NUMERIQUE"
...
$compte_destination$ = $compte_source$ => "COMPTES_IDENTIQUES"
TRUE => "OK"
```

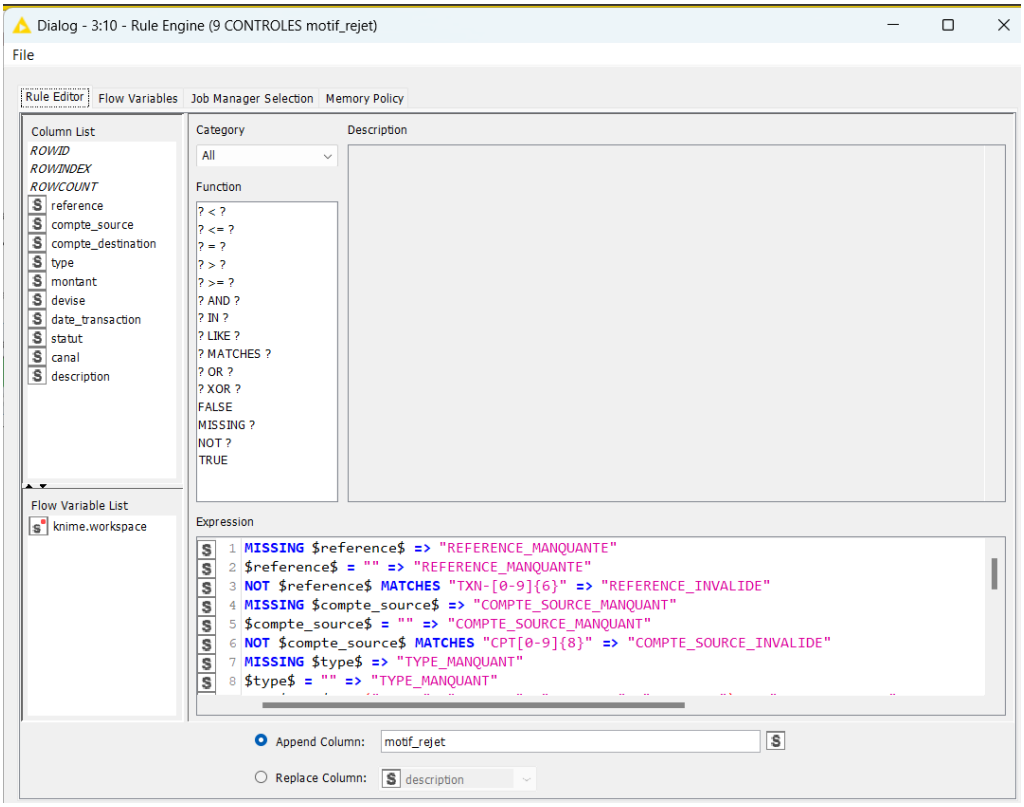


FIGURE 4.2 – Configuration du nœud 9 : les 30 règles de contrôle

4.4 Résultat obtenu

Les points de contrôle intermédiaires ont tous été atteints :

Tableau 4.2 – Points de contrôle du workflow (poste de SOMBIE, 29/10/2026)

Nœud	Contrôle	Lignes
1 CSV Reader	lignes lues, 10 colonnes en texte	1010
10 Séparation	sortie `OK` / rejets du nœud 9	956 / 54
13 Séparation	valides / doublons	946 / 10
23 GroupBy	motifs différents (total 64)	18

Tableau 4.3 – Rejets par motif produits par KNIME

Motif	Lignes	Motif	Lignes
DOUBLON_REFERENCE	10	DATE_INVALIDE	4
MONTANT_NEGATIF	5	REFERENCE_MANQUANTE	4
COMPTE_SOURCE_MANQUANT	4	TYPE_INVALIDE	4
COMPTE_SOURCE_INVALIDE	3	DATE_MANQUANTE	3
DESTINATION_MANQUANTE	3	DEVISE_INVALIDE	3
MONTANT_MANQUANT	3	MONTANT_NON_NUMERIQUE	3
MONTANT_NUL	3	REFERENCE_INVALIDE	3
STATUT_INVALIDE	3	CANAL_INVALIDE	2
COMPTES_IDENTIQUES	2	DESTINATION_INVALIDE	2
Total des rejets (54 anomalies + 10 doublons)			**64**

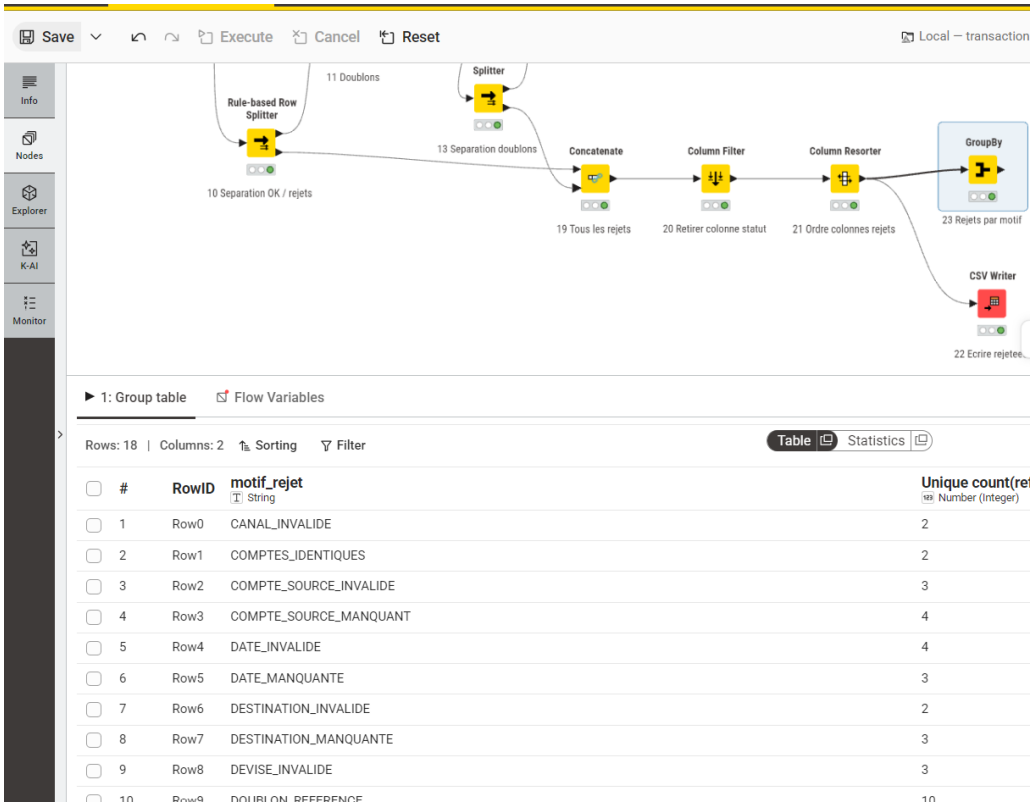


Figure 4.3 – Sortie du nœud 23 : 18 motifs de rejet, 64 lignes au total

4.5 Vérification automatique

La commande `npm run data:compare` compare la sortie de KNIME, ligne par ligne et champ par champ, au résultat de l'ETL de référence (`data/expected/`). Elle est obligatoire avant tout chargement.

Extrait 4.2 – Vérification de la sortie KNIME

```
$ npm run data:compare
[comparaison] CONFORME : 946 valides, 64 rejetées, motifs identiques au résultat attendu.
```

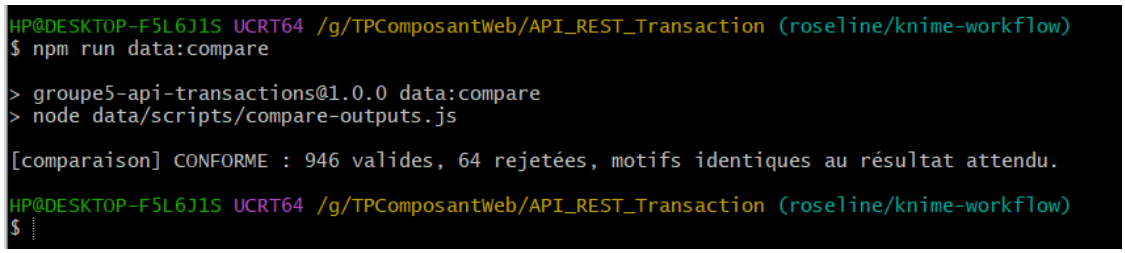


FIGURE 4.4 – La sortie de KNIME est conforme au résultat attendu

Pour rejouer la vérification sur un autre poste, on importe `knime/transactions_etl.knwf` dans KNIME, puis on adapte les chemins du CSV Reader (nœud 1) et des deux CSV Writer (nœuds 18 et 22).

4.6 Difficultés rencontrées

La construction a buté sur trois points. Aucun n'a demandé de modifier les règles : la règle du projet est de corriger le workflow, jamais les règles, pour « faire passer » la comparaison.

TABLEAU 4.4 – Difficultés rencontrées lors de la construction du workflow

Nœud	Symptôme	Cause et correction
2	17 colonnes au lieu de 10	le mode par défaut ajoute des colonnes suffixées `_transformed` ; choisir *Replace selected input columns*
12	règle en rouge	dans KNIME 5.12, la colonne de statut s'appelle `Duplicate Status` et non `duplicate-type-classifier` comme prévu ; insérer la colonne depuis la liste plutôt que la taper
18	le nœud refuse de s'exécuter	un port optionnel *File System Connection* avait été ajouté par erreur et restait non relié ; le retirer

La spécification `docs/KNIME_WORKFLOW.md` reste la référence pour reconstruire le workflow sur un autre poste. Son tableau « écart → cause → correction » couvre les erreurs que la comparaison peut signaler.

5. API REST de transactions

5.1 Responsabilités et organisation

L'API expose les transactions conservées dans PostgreSQL et les résultats du chargement KNIME. Elle est construite avec Express 5 et séparée en couches afin que les routes ne portent ni les règles métier ni les détails SQL :

1. les routes HTTP reçoivent la requête et choisissent le code de réponse ;
2. les modules de validation normalisent et contrôlent les entrées ;
3. le dépôt de transactions exécute les opérations sur PostgreSQL ;
4. un gestionnaire central transforme les erreurs en réponses JSON cohérentes.

Cette séparation permet de tester les routes avec un dépôt en mémoire, puis de vérifier les requêtes SQL séparément contre PostgreSQL (chapitre 7).

5.2 Routes et opérations

Les transactions utilisent leur référence métier `TXN-xxxxxx` dans l'URL. Cette clé reste stable entre les chargements, contrairement à un identifiant technique de base.

Tableau 5.1 – Principales routes de l'API

Méthode	Route	Comportement principal
GET	`/api/transactions`	Liste filtrée, paginée et triée ; renvoie les données et les métadonnées.
GET	`/api/transactions/statistiques`	Agrégats par type et devise, avec les filtres de recherche.
GET	`/api/transactions/{reference}`	Détail d'une transaction.
POST	`/api/transactions`	Création d'une transaction de source `API` ; réponse 201 et en-tête `Location`.
PATCH	`/api/transactions/{reference}`	Modification partielle, après revalidation de l'objet complet.
DELETE	`/api/transactions/{reference}`	Suppression ; réponse 204 en cas de succès.
GET	`/api/rejets` et routes associées	Consultation des rejets, des statistiques par motif et du dernier chargement.

5.3 Filtres, pagination et tri

La liste accepte des filtres par type, statut, canal, devise, catégorie, source, comptes, période et montant. Les filtres d'énumération peuvent recevoir plusieurs valeurs séparées par une virgule. Le paramètre `compte` recherche dans le compte source ou destination. La pagination utilise `page` et `limite` (20 par défaut, 100 au maximum). Le tri est limité à la date, au montant ou à la référence.

Un tri secondaire par référence départage les lignes ayant la même valeur de tri. Les pages restent ainsi stables entre deux requêtes. Les paramètres inconnus ou invalides renvoient 400 au lieu d'être ignorés silencieusement : une faute de frappe ne peut donc pas élargir accidentellement la recherche.

5.4 Validation et réponses d'erreur

Les champs sont normalisés (espaces retirés, codes en majuscules), puis contrôlés selon le contrat commun décrit au chapitre 3. Les règles portent notamment sur les formats de référence et de

compte, les valeurs autorisées, les dates réelles, les montants positifs et les cohérences entre champs. La catégorie de montant est calculée côté serveur. Les corps de requête inconnus sont refusés.

Pour un `PATCH`, l'API fusionne les champs fournis avec la transaction existante, puis valide l'ensemble. Un virement ne peut ainsi pas perdre son compte destination tout en restant accepté. Les erreurs de validation sont regroupées par champ dans une réponse 400. Les principaux statuts sont 400 (entrée invalide), 404 (ressource absente), 409 (référence déjà utilisée), 503 (base indisponible) et 500 (erreur inattendue). Le format commun contient un code stable, un message et, si nécessaire, le détail des champs fautifs.

5.5 Sécurité des requêtes SQL

Les valeurs reçues du client sont toujours transmises à PostgreSQL par paramètres `$1`, `$2`, etc. Elles ne sont pas concaténées au texte SQL. Comme les noms de colonnes ne peuvent pas être paramétrés, les champs de tri sont choisis dans une liste blanche. Cette approche réduit le risque d'injection SQL ; elle ne remplace pas l'authentification ni les contrôles d'accès.

5.6 Vérification

Les tests unitaires couvrent la validation et la génération des requêtes paramétrées. Les tests HTTP utilisent Supertest et un dépôt en mémoire pour contrôler les codes, les filtres, la pagination, les écritures et les erreurs sans dépendre de PostgreSQL. Des tests d'intégration complètent cette couverture sur une vraie base, et la collection Postman vérifie les parcours de démonstration. Les résultats effectivement exécutés sont consignés dans le rapport de tests du projet.

6. Supervision avec Prometheus et Grafana

6.1 Les métriques exposées

Un middleware Express mesure chaque requête et la bibliothèque `prom-client` publie les résultats sur `/metrics`, au format texte de Prometheus.

Tableau 6.1 – Métriques de l'API : requêtes, erreurs et temps de réponse (exigence E4)

Métrique	Type	Ce qu'elle mesure
`http_requests_total`	Counter	requêtes, par méthode, route et code HTTP
`http_request_errors_total`	Counter	erreurs, classées `client` (4xx) ou `serveur` (5xx)
`http_request_duration_seconds`	Histogram	temps de réponse, seaux de 5 ms à 5 s
`http_requests_in_flight`	Gauge	requêtes en cours
`transactions_operations_total`	Counter	créations, modifications, suppressions (métrique métier)

Un *Counter* ne fait que monter, une *Gauge* monte et descend, un *Histogram* range les durées par seaux pour calculer des centiles. Le label `route` contient le *modèle* de route (`/api/transactions/:reference`) et jamais l'URL réelle : sinon chaque référence créerait sa propre série et leur nombre exploserait.

6.2 Prometheus

Prometheus vient lire `api:3000/metrics` toutes les 5 secondes (modèle *pull*) sous le nom de tâche `api-transactions` : l'API n'a pas besoin de connaître Prometheus.

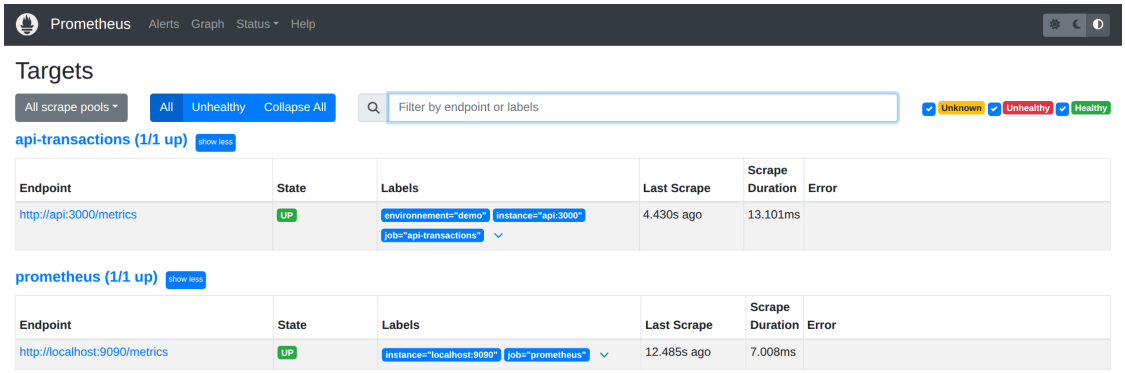


FIGURE 6.1 – Prometheus : la cible `api-transactions` est UP

Trois règles d'alerte sont chargées ; en fonctionnement normal, elles sont toutes inactives.

TABLEAU 6.2 – Règles d'alerte (`monitoring/prometheus/alert_rules.yml`)

Alerte	Condition	Durée
`ApiIndisponible`	la cible ne répond plus (`up == 0`)	30 s
`TauxErreursServeurEleve`	plus de 5 % d'erreurs serveur	1 min
`LatenceP95Elevee`	p95 au-dessus de 500 ms	2 min

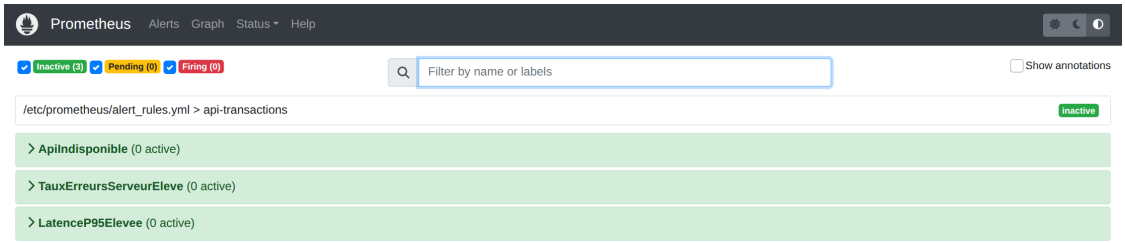


FIGURE 6.2 – Prometheus : les trois règles d'alerte, inactives

Requêtes PromQL principales :

```
sum(rate(http_requests_total{job="api-transactions"}[1m]))                    # débit
sum(rate(http_request_errors_total[5m])) / sum(rate(http_requests_total[5m]))   # taux d'erreurs
histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))  # p95
```

6.3 Grafana

Grafana se configure seul au démarrage, à partir des fichiers de `monitoring/grafana/` : source de données Prometheus et tableau de bord « API Transactions - Groupe 5 ». Ses 13 panneaux : API disponible, requêtes par seconde (globales et par route), taux d'erreurs, erreurs par code HTTP, latence p95, centiles p50/p95/p99, temps de réponse moyen par route, répartition des codes HTTP, écritures, requêtes en cours, mémoire du processus et latence de la boucle d'événements Node.js.

La figure 6.3 a été prise pendant le trafic de démonstration (`npm run demo:traffic -- -rps 6`), qui mêle lectures, créations, modifications, suppressions et erreurs volontaires (400, 404, 409). L'API traite environ 6 requêtes par seconde avec une latence p95 d'environ 46 ms. Le taux d'erreurs affiché (environ 15 %) correspond à ces erreurs *client* volontaires : aucune erreur serveur n'apparaît.



FIGURE 6.3 – Le tableau de bord pendant le trafic de démonstration

6.4 Panne contrôlée de la base

Pendant le trafic, nous avons arrêté PostgreSQL (`docker compose stop postgres`) pendant 85 secondes, de 17 :56 :49 à 17 :58 :14 le 29/10/2026, puis l'avons redémarré (`docker compose start postgres`). Observations :

- l'API reste vivante : `/health` répond 200, et `/health/ready` signale la panne (statut `degrade`, base `indisponible`) ;
- les routes de données répondent 503 `BASE_INDISPONIBLE` ; sur la fenêtre de 6 minutes de la figure 6.4, Grafana compte 408 réponses 503 et le taux d'erreurs monte à 50 % ;
- les temps de réponse p95 et p99 montent à près de 5 s : c'est le délai d'attente de connexion à la base, après lequel l'API répond 503 au lieu de rester bloquée ;
- au redémarrage de la base, l'API se reconnecte seule et répond de nouveau 200, sans avoir été relancée.



FIGURE 6.4 – Grafana pendant la panne de PostgreSQL : réponses 503 et délais de connexion

7. Tests et résultats

7.1 Stratégie

Aucune interface web n'est fournie : conformément à l'énoncé (E6), nous utilisons des outils de test d'API plutôt que Cypress. Les tests sont organisés du plus rapide au plus complet :

TABLEAU 7.1 – Les suites de tests

Suite	Tests	Ce qui est vérifié
Données (`node:test`)	9	déterminisme ; chaque anomalie injectée donne le motif attendu ; normalisation ; doublons ; comparateur
Validation (Jest)	42	corps POST et PATCH, paramètres de requête, dates réelles, catégories
SQL (Jest)	5	paramètres `$n`, tentative d'injection, liste blanche du tri, pagination
Chargeur (Jest)	5	conversion des nombres KNIME, revalidation, doublons, catégorie, BOM
API HTTP (Jest, Supertest)	28	codes 200, 201, 204, 400, 404, 409, 503 ; filtres ; pagination ; `/metrics`
Intégration PostgreSQL	9	schéma réel, chargement, idempotence, contraintes, filtres, CRUD
Postman (Newman)	15	le scénario de démonstration, erreurs comprises
Bout en bout	7	toute la chaîne, des données à Grafana

7.2 Le test de bout en bout

L'énoncé exige un test de bout en bout de la chaîne principale. `tests/e2e/chaine-complete.test.js` enchaîne sept vérifications ; le numéro de celle qui échoue désigne le maillon en défaut :

1. l'API et la base répondent ;
2. les données chargées sont cohérentes avec le fichier produit (et avec le résultat attendu si elles viennent de KNIME) ;
3. les filtres et la pagination renvoient des résultats corrects ;
4. un cycle complet création, lecture, modification, suppression, avec les erreurs attendues ;
5. `/metrics` expose requêtes, erreurs et temps de réponse ;
6. Prometheus collecte les métriques de l'API ;
7. Grafana est provisionné (source Prometheus et tableau de bord).

```
like-code@like-code-Hamidou:~/Bureau/API_REST_Transaction$ npm run test:e2e

> groupe5-api-transactions@1.0.0 test:e2e
> node --test --test-concurrency=1 "tests/e2e/*.test.js"

✔ 1. l'API et la base répondent (62.996089ms)
✔ 2. les données KNIME sont chargées et cohérentes avec le fichier produit (19.801475ms)
✔ 3. les filtres et la pagination renvoient des résultats corrects (36.852071ms)
✔ 4. cycle d'écriture complet et erreurs attendues (48.006162ms)
✔ 5. /metrics expose requêtes, erreurs et temps de réponse (7.03032ms)
✔ 6. Prometheus collecte les métriques de l'API (2550.785664ms)
✔ 7. Grafana est provisionné (source Prometheus + tableau de bord) (92.222456ms)
ℹ tests 7
ℹ pass 7
ℹ fail 0
```

FIGURE 7.1 – Test de bout en bout : 7 vérifications réussies

7.3 Résultats obtenus

Chaque ligne correspond à une commande réellement exécutée.

TABLEAU 7.2 – Résultats des tests

Commande	Résultat	Poste, date
`npm run test:data`	pass 9, fail 0	BANDE, 28/10
`npm run data:reference`	946 valides, 64 rejetées	BANDE, 28/10
`cd api && npx jest`	80 passed (+ 9 ignorés sans base)	BANDE, 29/10
`curl /health/ready`	`{"statut":"ok","base":"ok"}`	BANDE, 29/10
INSERT invalide	refusé par `CHECK`	BANDE, 28/10
Chargement ×2	946 puis 0 insérées	BANDE, 28/10
`npm run test:integration`	9 passed	BANDE, 29/10
Newman (collection Postman)	15 requêtes, 15 assertions, 0 échec	BANDE, 29/10
`npm run test:e2e`	pass 7, fail 0	BANDE, 29/10
`npm run data:compare` (sortie KNIME)	CONFORME : 946 valides, 64 rejetées	SOMBIE, 29/10
Panne de PostgreSQL (85 s)	503 pendant la panne, 200 après, sans relancer l'API	BANDE, 29/10
Scripts PowerShell (6 fichiers)	0 erreur de syntaxe (PowerShell 7.4)	BANDE, 29/10
Scripts PowerShell sous Windows (Docker Desktop)	`demarrer`, `charger-donnees` (946 insérées), `tester-tout` : 4 REUSSI	SOMBIE, 06/11
Intégration, Newman, E2E sous Windows	9 passed ; 15 requêtes, 0 échec ; pass 7	SOMBIE, 06/11

Les tests sous Docker ont été joués avec les données de l'ETL de référence (`data/expected`). La sortie de KNIME lui est identique ligne à ligne (`npm run data:compare` : CONFORME), ce qui garantit le même résultat avec les données produites par KNIME.



FIGURE 7.2 – Collection Postman jouée avec Newman : 15 requêtes, 15 assertions, 0 échec

8. Installation et reproductibilité

8.1 Objectif

L'énoncé demande une installation reproductible sur une machine neuve. Le guide `docs/INSTALLATION_WINDOWS.md` décrit cette installation pas à pas, pour un débutant sous Windows 10 ou 11. Il faut compter 45 à 90 minutes la première fois, surtout pour les téléchargements.

Deux familles d'outils sont installées différemment :

— **les services** (PostgreSQL, API, Prometheus, Grafana) tournent dans Docker : une seule commande les construit et les démarre, avec les mêmes versions sur tous les postes ;

— **les applications de bureau** (Node.js, Git, Docker Desktop, KNIME) s'installent à la main. KNIME n'est nécessaire que sur le poste chargé de l'ETL.

8.2 Prérequis

Tableau 8.1 – Outils à installer sur le poste

Outil	Version	Vérification
Node.js	22 LTS ou plus	`node -v`
Git (avec Git Bash)	2.x	`git -version`
Docker Desktop (WSL 2)	Compose v2	`docker compose version`, puis `docker run -rm hello-world`
KNIME Analytics Platform	5.x	s'ouvre ; poste ETL seulement

Le script `scripts/verifier-prerequis.ps1` contrôle ces outils et les ports utilisés (5432, 3000, 9090, 3001). Un port déjà pris se change dans le fichier `.env`, sans toucher au code.

8.3 Les étapes

1. vérifier Node.js et Git ;
2. installer WSL 2 et Docker Desktop, puis attendre l'état *Engine running* ;
3. cloner le dépôt et créer le fichier `.env` à partir de `.env.example` ;
4. lancer `scripts/verifier-prerequis.ps1` ;
5. démarrer la pile avec `scripts/demarrer.ps1` (environ 1,5 Go d'images au premier lancement) ;
6. vérifier chaque service : API, `/health/ready`, `/metrics`, cibles Prometheus, tableau de bord Grafana, tables PostgreSQL ;
7. sur le poste ETL, installer KNIME, importer le workflow et vérifier sa sortie avec `npm run data:compare` ;
8. charger les données avec `scripts/charger-donnees.ps1` ;
9. lancer tous les tests avec `scripts/tester-tout.ps1`.

Chaque étape du guide donne le résultat attendu et un tableau « message d'erreur → cause → correction ». On ne passe à l'étape suivante que si le résultat attendu est obtenu.

Tableau 8.2 – Arrêter, relancer, réinitialiser

Besoin	Commande	Données
Arrêter	`scripts/arreter.ps1`	conservées
Relancer	`scripts/demarrer.ps1`	conservées, pas de rechargement
Tout remettre à zéro	`scripts/reinitialiser.ps1`	supprimées : recharger ensuite

8.4 Pièges constatés sur les postes de l'équipe

Tableau 8.3 – Problèmes rencontrés et corrections

Symptôme	Cause et correction
`npm` échoue avec `Cannot find module ...jest.js`	chemin du projet contenant un espace, un accent ou `&` ; utiliser un chemin simple comme `C:\projets\...`
`cd G:\TPComposantWeb\...` : *No such file or directory* dans Git Bash	Git Bash supprime les barres obliques inverses ; écrire `/g/TPComposantWeb/...`
L'export KNIME n'est pas dans le dépôt	KNIME propose par défaut le dossier personnel ; choisir `knime\transactions_etl.knwf` dans le dépôt
Montants vides ou `?` dans KNIME	colonnes non forcées en texte dans le CSV Reader
Le *workspace* KNIME apparaît dans `git status`	le choisir hors du dépôt, par exemple `C:\Users\<nom>\knime-workspace`

8.5 Test sur un poste neuf

Le guide est validé en le suivant pas à pas, dans un dossier vide, sur un clone neuf du dépôt et avec des volumes Docker neufs (base vide au départ). Faute de poste Windows vierge disponible avant la soumission, le test a été mené le 29/10/2026 sur le poste de BANDE (Linux), en suivant les commandes du `README.md`. Chaque blocage rencontré est noté avec sa correction. La procédure Windows a ensuite été jouée le 06/11/2026 sur le poste de SOMBIE (Windows 11, Docker Desktop installé pour l'occasion) avec les scripts PowerShell du dépôt : pile démarrée, 946 transactions chargées et toutes les suites de tests réussies (chapitre 7).

Tableau 8.4 – Déroulé du test d'installation sur un clone neuf (poste de BANDE, 29/10/2026)

Étape du guide	Résultat	Durée
1–2 Node.js, Git, Docker	Node.js 24.20, npm 11.19, Git 2.43, Docker 29.8 avec Compose v2	< 1 s
3–4 Clonage, prérequis	clone dans un dossier vide ; `.env` créé depuis `.env.example`	4 s
5–6 Pile Docker et services	1er essai refusé : port 5432 déjà utilisé ; avec `POSTGRES_HOST_PORT=5433`, 4 services démarrés, 3 volumes neufs, `/health/ready` OK	9 s + 8 s
7 KNIME et `data:compare`	CONFORME : 946 valides, 64 rejetées (poste de SOMBIE, chapitre 4)	–
8 Chargement	base vide au départ ; 1er passage 946 insérées, 64 rejets ; 2e passage 0 insérée, 946 mises à jour	2 s
9 Tests	`test:data` pass 9 ; `npx jest` 80 passed ; intégration 9 passed ; `test:e2e` pass 7	10 s

Bilan. Hors téléchargements, l'installation complète et la vérification prennent moins d'une minute. Les images Docker étaient déjà présentes sur le poste : sur une machine réellement neuve, leur premier téléchargement ajoute quelques minutes selon la connexion. Un seul blocage a été rencontré : le port 5432 déjà occupé par un PostgreSQL local. Il se corrige en changeant la seule ligne `POSTGRES_HOST_PORT` du fichier `.env`, comme l'indique le guide. L'étape KNIME n'a pas été rejouée sur ce poste (application de bureau, procédure distincte) : sa sortie, identique au résultat attendu, est vérifiée par `npm run data:compare`.

9. Limites et perspectives

Le projet démontre une chaîne intégrée de traitement et d'exposition de transactions fictives. Il s'agit d'un livrable pédagogique, pas d'un système prêt à traiter des opérations bancaires réelles. Les limites suivantes doivent être prises en compte.

9.1 Périmètre fonctionnel et données

Les données sont entièrement synthétiques et produites par un générateur déterministe. Elles permettent de reproduire les tests, mais ne représentent ni la diversité ni les contraintes de confidentialité de données bancaires réelles. L'API ne réalise pas de mouvement de fonds : elle enregistre et expose des objets de transaction.

Le workflow KNIME est conçu pour une exécution sur poste de travail et une production de fichiers CSV. Cette étape reste distincte de l'API et de Docker. Pour une exploitation régulière, il faudrait automatiser son exécution, superviser les fichiers produits et définir une stratégie de reprise en cas d'échec de l'ETL.

9.2 Sécurité et exploitation

L'API ne comporte pas d'authentification, d'autorisation par rôle ni de limitation de débit. Les services et identifiants fournis par défaut sont destinés au développement local. Avant toute exposition sur un réseau non maîtrisé, il faudrait ajouter une gestion d'identité (par exemple une clé d'API ou des jetons), protéger les secrets, configurer TLS et définir une politique d'accès minimale à PostgreSQL.

Prometheus collecte les métriques et Grafana les présente, mais les règles d'alerte ne disposent pas d'un service de notification tel qu'Alertmanager. Les journaux sont consultables dans les sorties des conteneurs ; ils ne sont pas centralisés ni conservés dans un système de recherche. La pile Docker Compose vise un seul poste et ne fournit ni haute disponibilité, ni sauvegarde automatisée, ni stratégie de déploiement multi-instance.

9.3 Évolutions possibles

Les suites naturelles sont l'automatisation du workflow KNIME, l'ajout d'une authentification et d'une autorisation adaptées, la gestion centralisée des secrets, les sauvegardes/restaurations testées, ainsi que l'envoi des alertes et la centralisation des journaux. Une mise en production nécessiterait également des tests de charge plus représentatifs, une politique de rétention des métriques et journaux, et une revue de sécurité de l'ensemble de la chaîne.

A. Annexes

A.1 Contrat de données d'une transaction

Champ	Format	Règle
`reference`	`TXN-` + 6 chiffres	obligatoire, unique
`compte_source`	`CPT` + 8 chiffres	obligatoire

Champ	Format	Règle
compte_destination	CPT + 8 chiffres	obligatoire pour un virement ; différent du compte source
type	énumération	DEPOT, RETRAIT, VIREMENT, PAIEMENT
montant	nombre, 2 décimales	strictement positif
devise	énumération	XOF (défaut), EUR, USD
date_transaction	AAAA-MM-JJThh:mm:ss	date réelle
statut	énumération	EFFECTUEE, EN_ATTENTE, ANNULEE
canal	énumération	GUICHET, GAB, MOBILE, INTERNET
description	texte	255 caractères au plus, facultatif
categorie_montant	calculé	FAIBLE, MOYEN, ELEVE
source	calculé	KNIME ou API

Tableau A.1 – Contrat de données (docs/CONTRATS.md)

A.2 Organisation du dépôt

```
api/            API Express : src/, tests/, Dockerfile, openapi.yaml
data/           générateur, règles, données brutes, résultat attendu, sortie KNIME
db/init/        schéma SQL
knime/          workflow KNIME exporté (.knwf)
monitoring/     configurations Prometheus et Grafana
tests/          test de bout en bout, collection Postman
scripts/        scripts PowerShell, générateur de trafic, contrôle de cohérence
docs/           exigences, architecture, contrats, installation, KNIME, démonstration, tests
rapport/        ce rapport (LaTeX)
```